Research and Adaptation of Components from the Persistent OS Phantom for Genode

student: Mikhail Voronin
supervisor: Artem Burmyakov
consultant: Alexander Tormasov

June 2026

Relevance

With the advent of new hardware, new demands are being placed on software, especially basic software. In light of this, it is necessary to examine the relevance of persistence concepts in the current environment.

Objective

There are already existing implementations of persistent systems designed for embedded systems and systems without constant physical human access. Our objective is to understand what data security options are available in such implementations.

Data Persistence

Persistence the period of time during which an object is usable
M. P. Atkinson, P. J. Bailey, K. J. Chisholm, W. P. Cockshott, and R. Morrison,
"Ps-algol: A language for persistent programming," (1983)

Today's dominant model (for non-persistent programming) — **two-level store**:

- Data is either *active* (RAM) or *stored* (disk)
- Programmer manually moves data between levels
- Consequences: data loss, I/O boilerplate (≈30% of code)

Orthogonal Persistence

Alternative: the **environment** manages persistence

Three principles of **orthogonal persistence** (*according to R. Morrison, M. P. Atkinson — "Persistent Languages and Architectures", 1990*):

1. **Independence** — data persistence does not depend on how it is manipulated
2. **Data type orthogonality** — all types can be persisted
3. **Identification** — type does not affect persistence identity

Benefits: No initialization is required at every system startup; there is no need for regular configuration reads/writes; up to 30% reduction in code size; more efficient use of disk bandwidth; improved reliability failt tolerance.

PhantomOS

PhantomOS is a non-Unix like OS implementing orthogonal persistence (D. Zavalishin, 2010).

- Persistent Virtual Machine (PVM): bytecode VM + POSIX layer
- Snapshots saved periodically via **Copy-on-Write**
- Only changed pages written to disk (incremental)
- Programs paused only briefly during snapshot preparation

Currently ported to the **Genode** OS C++ framework:

- Proven microkernels (NOVA, seL4, Fiasco.OC)
- Capability-based security, minimal TCB

Current snapshot mechanism: **Snapper**

Snapper

Snappser is a file-system-based generational snapshot storage

R. Mitov, "Snapper (v2): A PhantomOS snapshot mechanism," FOSDEM 2026

Previous approach ("superblock"): snapshots stored as monolithic objects.

+ Fast
– No fault tolerance, no storage recovery

Snapper 2.0:

- Unchanged pages **linked**, not copied across generations
- ext4 + fsck for crash recovery
- Integrity checks (hashes per file)
- Configurable redundancy policy

Problem Statement

PhantomOS snapshots capture the **entire machine state** — including sensitive data present in memory.

Snapshots are stored on the drive in **plaintext**. An attacker with physical access can mount and read them directly, or fully reproduce the machine state.

- No prior research on this for PhantomOS
- No ready-made solution exists in Genode

Goal: implement a mechanism for the secure storage of snapshots.

Environment & Requirements

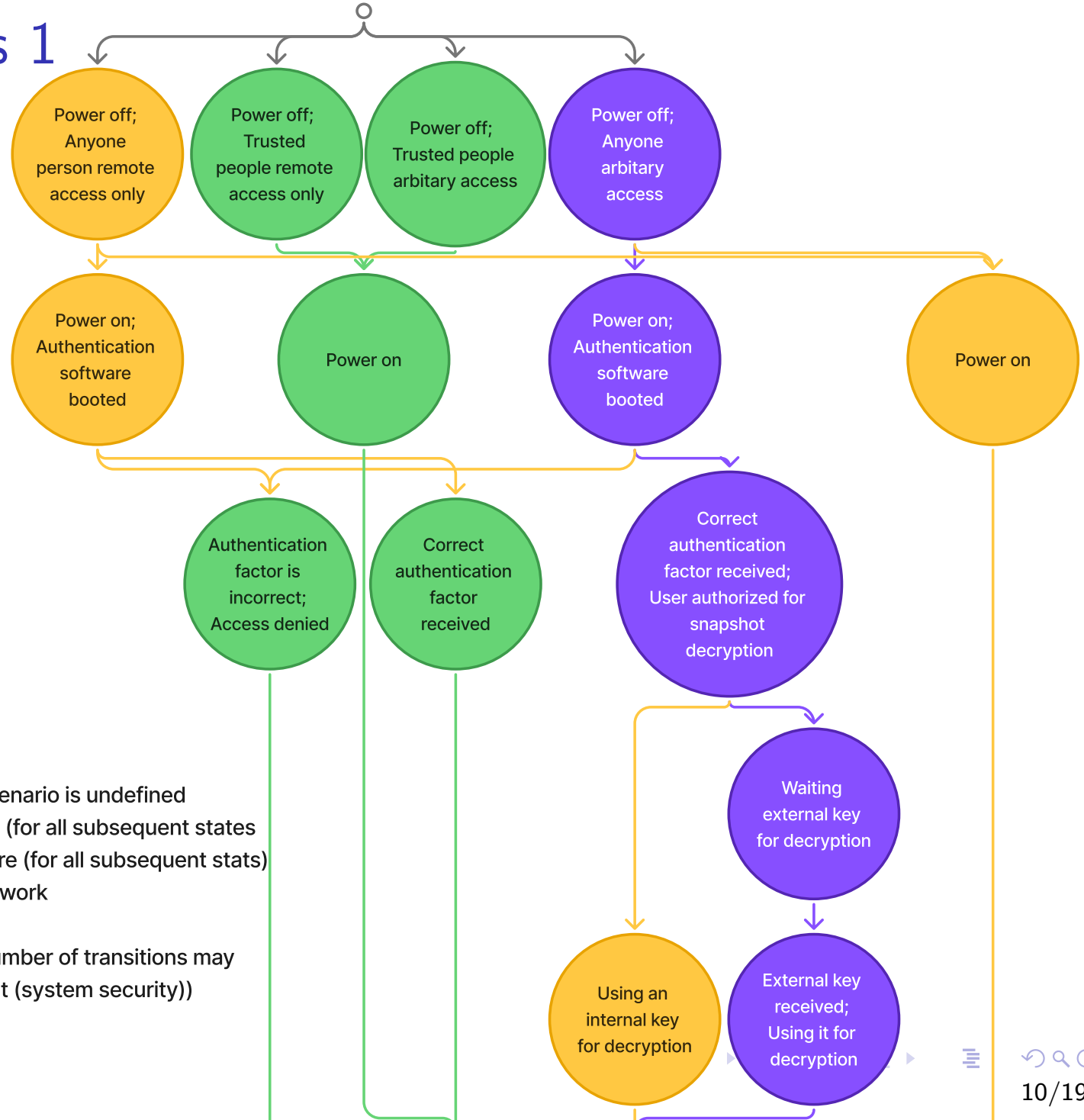
There should be no way for an attacker to gain access to the snapshot in plaintext at any point during its lifetime.

Requirements:

1. Mandatory authorization
2. Key stored on an **external USB drive**
3. Encryption of snapshots (AES, block-level)
4. Only encrypted data written to disk
5. Acceptable performance overhead ($\leq 2\times$)

Requirements 1 & 2 combined: the USB drive contains the key and serves as the authentication factor.

Usage scenarios 1



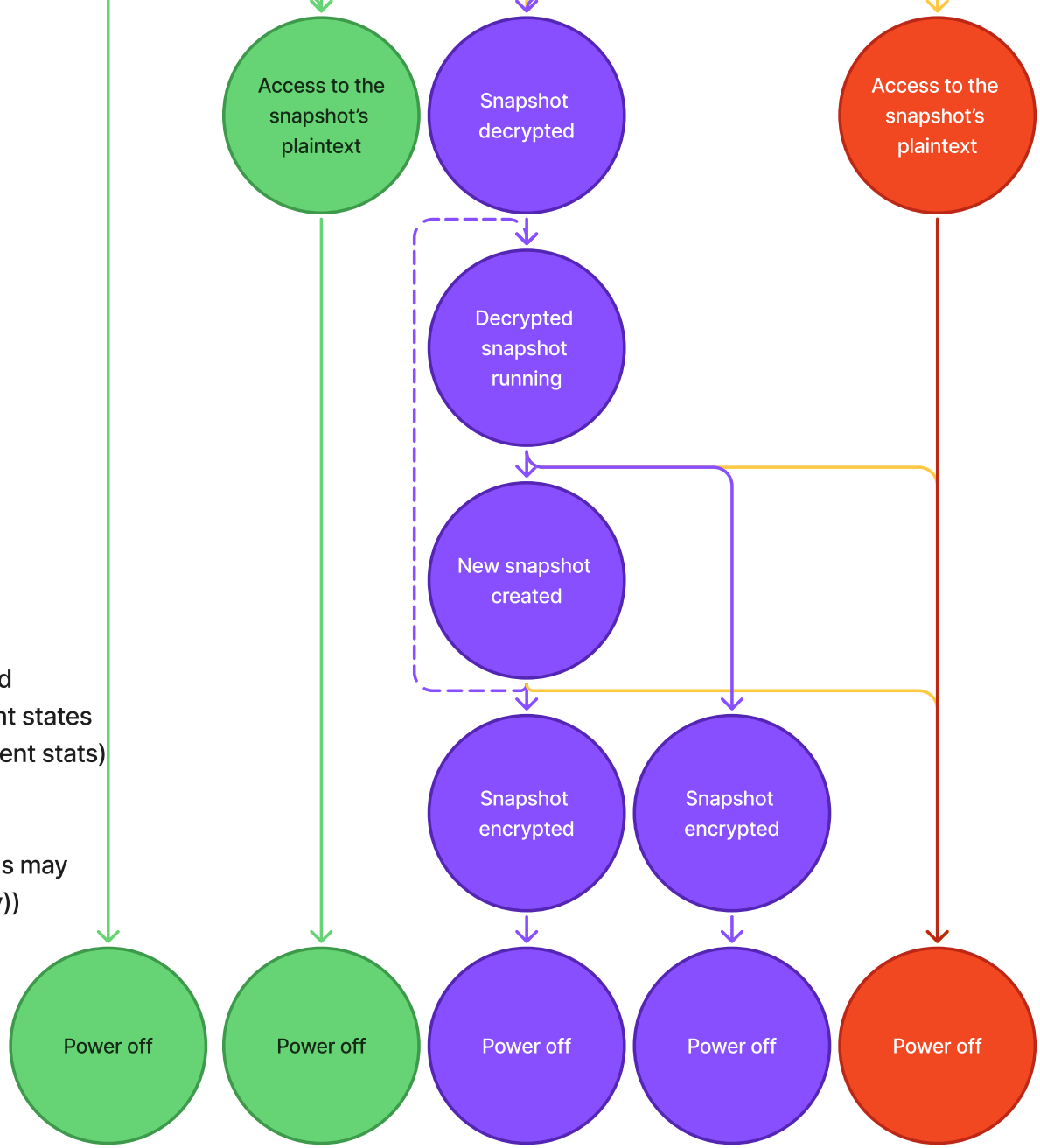
Legend:

- System state (computer - user)
- Note to the state
- In this state, the security of the scenario is undefined
- In this state the scenario is secure (for all subsequent states
- In this state the scenario is insecure (for all subsequent stats)
- This scenario is considered in our work
- Transition between states
- Transition between states (The number of transitions may vary and does not affect the final result (system security))

Usage scenarios 2

Legend:

- System state (computer - user)
- Note to the state
- In this state, the security of the scenario is undefined
- In this state the scenario is secure (for all subsequent states
- In this state the scenario is insecure (for all subsequent stats)
- This scenario is considered in our work
- Transition between states
- Transition between states (The number of transitions may vary and does not affect the final result (system security))



Implementation: se_vault

There is a **file_vault** in Genode based on **Tresor** library:

- Tresor: AES encryption on 4K blocks, SHA-256 integrity, CoW atomicity
- file_vault: state orchestrator providing a secure FS session

Implemented **se_vault** as adapted **file_vault**:

- Headless operation
- Passphrase replaced by **USB key file**
- rump_fs/ext2 replaced by **lwext4/ext4**
- Added **fsck** crash recovery support with fsck
- Direct block session (skip image file creation)
- All block sizes standardized to 4K
- Graceful shutdown sequence added

Related Fixes & Adaptations

For se_vault to function properly, lwext4, isomem, snapper, and vfs have been adapted.

lwext4 bug fixes:

- `ftruncate`: pointer cast for correct file size
- `sync` call now propagated to lower VFS layers (critical — Tresor updates its state hash on `sync`)

Shutdown sequence (isomem / snapper / VFS):

- isomem: power-off button triggers final snapshot
- snapper, vfs: client counters trigger graceful shutdown
- lwext4 unmounted cleanly; mount flag cleared for fsck

Optimization: skipping image file pre-allocation
reduced initialization time from ≈20 min to ≈1 min.

Benchmarking configuration

Environment: QEMU with `-cpu host` (enables AES/SHA hardware acceleration).
2 GB image for isomem, 5 GB image for se_vault (4 GB user FS).

Functional testing

Environment: QEMU with `-cpu host` (enables AES/SHA hardware acceleration).
2 GB image for isomem, 5 GB image for se_vault (4 GB user FS).

Functional tests (all passed):

- Normal boot / shutdown / restore cycle
- Encrypted image cannot be mounted without key
- Key/hash replacement attack — superblock not detected
- Crash recovery via fsck

Performance: first snapshot creation and restore, 5 runs each.

Performance testing

Environment: QEMU with `-cpu host` (enables AES/SHA hardware acceleration).
2 GB image for isomem, 5 GB image for se_vault (4 GB user FS).

Functional tests (all passed):

- Normal boot / shutdown / restore cycle
- Encrypted image cannot be mounted without key
- Key/hash replacement attack — superblock not detected
- Crash recovery via fsck

Performance: first snapshot creation and restore, 5 runs each.

Results

Operation	Without enc.	With enc.	Overhead
Snapshot creation	2.10 s	51.00 s	$\approx 24\times$
Snapshot restore	2.50 s	27.30 s	$\approx 11\times$

Root cause: **known Tresor library limitation** — not optimized for multithreading. Acknowledged by the author; unresolved.

Evaluation

4 of 5 requirements met.

- ✓ Mandatory authorization (USB key)
- ✓ Key on external medium
- ✓ AES block-level encryption
- ✓ Only encrypted data on disk
- × Overhead $\leq 2\times$ *(Tresor limitation)*

Security gain: snapshot cannot be mounted or read without the key.
Integrity: SHA-256 at block level (Tresor) + hash at FS level (Snapper).

Conclusion

Adapted **file_vault** $\rightarrow$ **se_vault** for encrypted snapshot storage in PhantomOS on Genode.

First security research specifically for PhantomOS.
No analogous component existed prior to se_vault.

Future work:

- Fix Tresor performance (or switch to FS-level encryption via OpenSSL)
- Encrypt the isomem block device using se_vault
- Investigate merging Snapper and Tresor snapshot concepts